

2026-04-15- 讨论

当前进展、问题收束 与下一步投稿判断

这次汇报只做三件事：给出现状判断、解释为什么先收束 project_1、以及明确需要拍板的问题。

project_1 优先级

pyfcstm vs
pyudbm

control-state 定义

博士研究进展对齐

project_1

建模主线 / 当前优先级最高

project_2

性质与场景生成 / 接口层

project_3

verification backend / 地基已起

project_4

iterative repair / 依赖前置成熟

日期 2026-04-15

明天这次讨论，我最希望先对齐三个决策

先拍板问题边界，再决定论文如何写和接下来 6 周怎么推

01 主投稿先锁定

project_1

先把主稿打成型，不再一次性拉四个 project。

02 对象先限于离散控制状

态层

先把模式、阶段、互锁、恢复、局部定时这一层讲透。

03

先讲清 pyfcstm / pyudbm 分工

一个回答目标对象，一个沉淀 timed backend。

后续展开顺序

1. 结论先给

2. project_1 文库证据

3. 基础设施与仓库进展

4. 拍板事项与倒排计划

一句话主线

不是再去找更多材料，而是让现有材料收敛成一篇问题边界清楚、对象清楚、基础设施落点清楚的会议论文。

当前最缺的不是材料，而是把问题对象收束清楚

一页结论先给出来，后面逐页用文库与仓库证据补强

1 先锁定 project_1

当前第一优先级是把主稿收束成型。

2 真正缺的是收束

project_1 当前不缺样本，缺的是问题定义。

3 先做离散控制状态层

目标对象先落到模式、阶段、互锁、恢复、局部 timer。

4 pyfcstm 是研究基座

它应被写成 control-state 基础设施。

5 project_3 有地基

pyudbm 已推进很深，仍缺核心搜索。

6 反馈基础设施最关键

LLM-based modeling 的杀手锏是持续反馈，而不只是 prompt。

Take-home

先收束对象，再拉厚实验；只要定义稳住，现有文库和仓库推进已经足以支撑一篇面向下一轮 conference 的主稿。

官方 2026 日程显示：A/B 主窗口已过，近端出口主要是 ESEM 与期刊

总体判断

main-track full paper 基本结束；近端只剩 ESEM、NIER 与 rolling journal

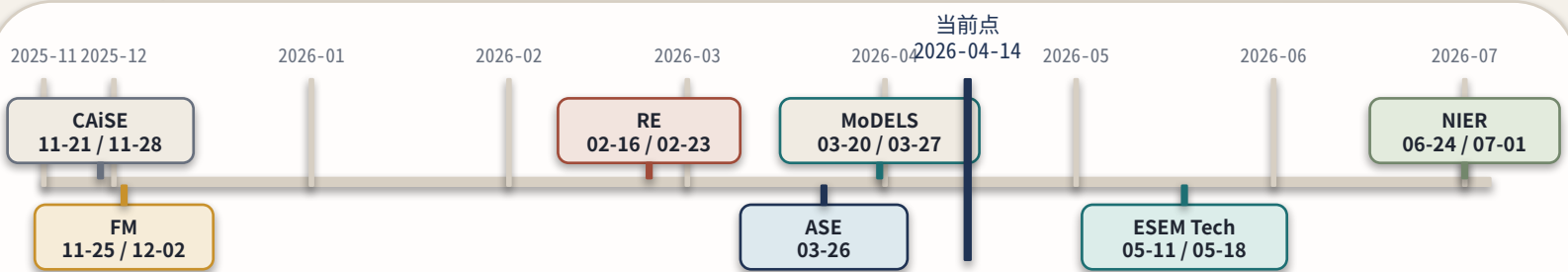
前因

CAISE / FM / RE / ASE / MoDELS research 等 A/B 主窗口都早于 2026-04-14。

因此

所以现在更该把 project_1 打成下一轮主稿，并把 ESEM / rolling journal 只当近端出口。

官方 2026 A/B 会议窗口（AoE）

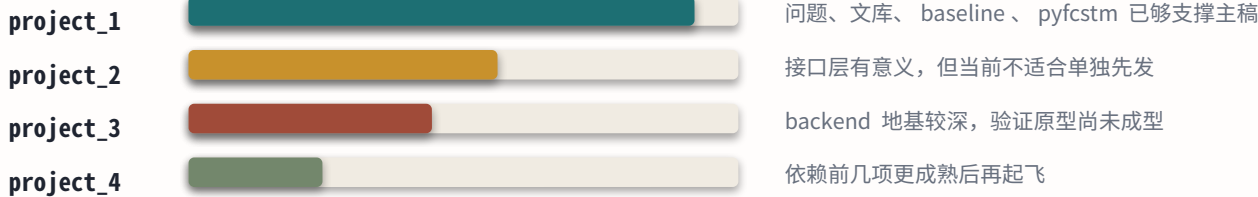


仍可操作窗口

路径	窗口
ESEM Tech	05-11 / 05-18
ESEM EVR	05-22 / 05-29
MODELS NIER	06-24 / 07-01

ISSRE research 04-17；若 04-10 摘要未交，基本不算现实窗口。

当前准备度



官方主页确认仍可投稿的期刊

TSE: CFP / submission
SoSyM: submit your manuscript
Requirements Engineering: submit your manuscript
ASE Journal / EMSE: submit your manuscript

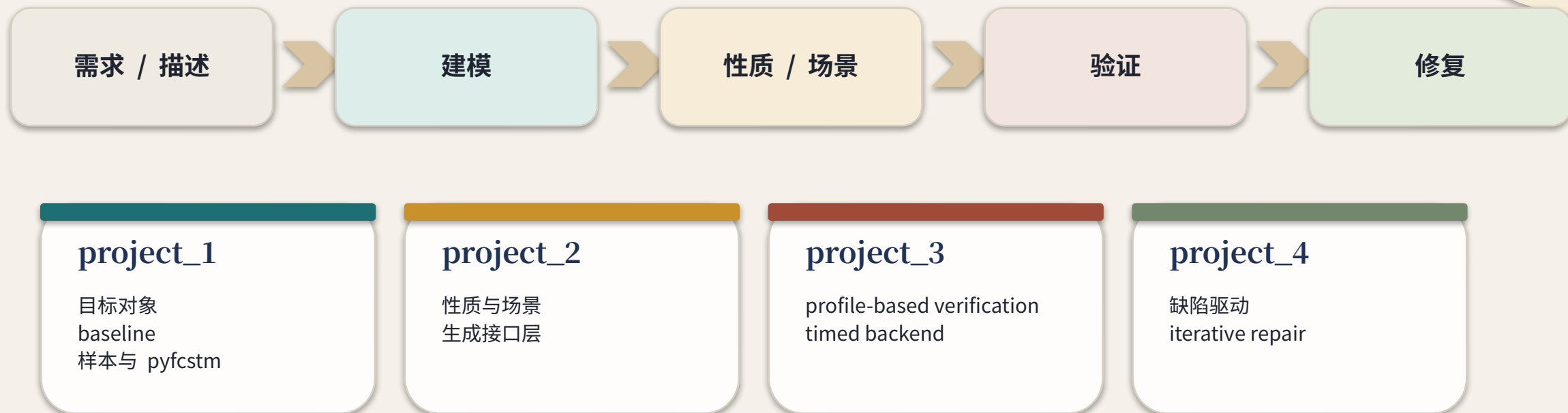
这些路径更适合作为近端外部输出，不应反过来牵引问题定义。

页面结论

现实的 this-semester 出口主要是 ESEM 或 rolling journals；但 project_1 仍应按下一轮主稿质量来打，而不是为了赶窗口扩题。

四个 project 是一条闭环，但近端主线必须先落在 project_1

先把建模对象和目标形式主义立住，后面的 scenario、verification、repair 才有共同基座



当前判断

博士主线仍然是生成 - 验证 - 修复闭环；只是本学期要先把建模对象与目标形式主义打稳，后续几项才有共同基座。

project_1 的说服力来自三条文库线加上一条基础设施线

不是平行摆设，而是共同回答“该建什么、凭什么、如何落地”

前因

project_1 不是靠单一材料就能站住，它需要比较对象、真实样本、状态机选型和可执行落地一起支持。

因此

所以这四条线必须被讲成一条证据链。

baseline 比较集

62 篇比较集
回答“该和谁比”

真实样本库

787 篇 / 746 条正例
回答“数据从哪里来”

状态机家族库

669 + 10 条目
回答“到底选哪类状态机”

pyfcstm 基础设施

executable IR
回答“如何落地”

中央结论
三条文库线共同把论文对象
压向 control-state 主问题

而 pyfcstm 则把这个对象变成可执行、可反馈、可持续迭代的模型空间。

绿色 baseline 已经勾出了 5 条可直接讨论的方法线

明天不该只报数量，而要把最值得讲的几篇“怎么做”讲清楚

62

baseline 条目

比较对象已经够用

状态分布



不是继续堆数量，而是把几条最硬的方法线讲清楚。

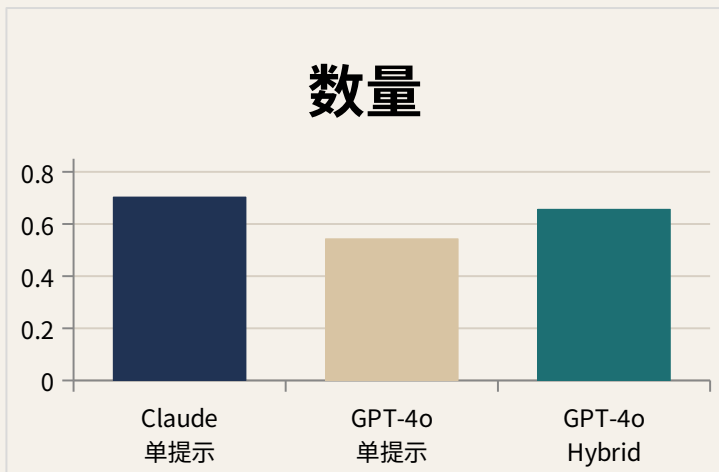
代表论文	方法骨架	对 project_1 的启发
Structure/Event-Driven 2026	多步结构分解 + 事件分解 + Hybrid 细化	direct baseline 已经出现，问题不再是“有没有人做”
SysML empirical 2025	两阶段 prompting + model checking feedback repair	真正有效的是 feedback loop，而不是一次性 prompt
IEC 61499 2025	iterative refinement + simulator / code generation	一旦接上仿真与部署，方法性质就变了
TTool AI 2024	知识注入 + 工具链反馈 + MBSE 集成	基础设施与领域知识会明显抬高效果

页面结论：baseline 已经不是“有没有”，而是“该把哪几类方法差异和反馈路线讲清楚”。

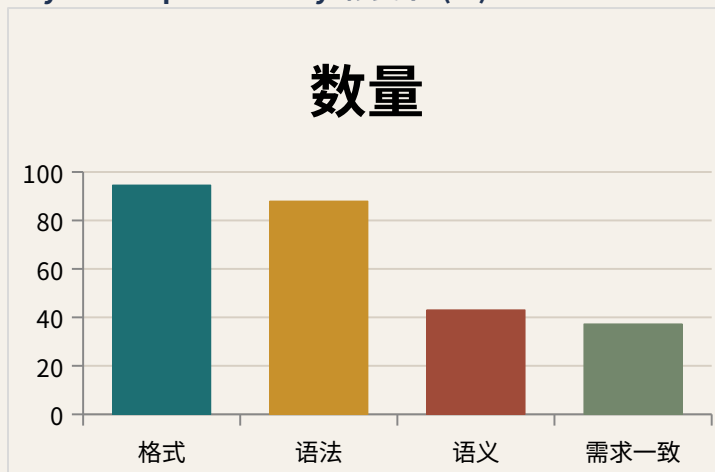
baseline 的共同趋势不是更花的 prompt，而是更强的工具反馈

几篇最值得讲的绿色条目给出的实证指向相当一致

2026 direct baseline F1



SysML empirical study 修复率 (%)



TTool AI

状态机评分 63 vs 58

速度快 15.2x

块图评分 81 vs 70

速度快 67.5x

IEC 61499 / workflow principles

一旦接上仿真、代码生成和 sanity checks，方法论就不再是一次 prompt。

Take-home

纯 prompt 可以给出骨架，但真正把质量拉上去的，是 model checking、仿真、编译检查和 workflow feedback。

sources/ 不是自然分布样本，而是面向数据集建设的治理后主集

如果不先说明这一点，后面的统计就很容易被误读

前因

如果不先讲治理口径，后面所有 EFSM/HSM/T0/T1 的比例都会被误读成自然分布。

因此

所以这一页先解释筛选逻辑，再看统计。

Warning

后面所有 EFSM / HSM / T0 / T1 的比例，首先反映的是收录策略与数据集治理目标，其次才是领域文献现象本身。

治理流程

广撒网检索

先把真实控制系统状态逻辑的文献面铺开。

标准化治理

按论文级、案例级、主类型、时间级别等口径做筛。

主集保留

优先保留 EFSM/HSM、T0/T1、细节高、不强趋同的案例。

五套治理口径

治理目标

优先保留 EFSM/HSM。
优先保留 T0/T1。
优先保留细节充实、可转为可执行样本的案例。
对强趋同簇做降采样控制。

口径	作用
论文级可用性	判断单篇论文整体上能否形成可靠样本
案例级角色	区分核心保留、清洗后保留、降采样保留
细节充实度	判断是否足以支撑高质量建模与评测
主类型	回答“默认应把它理解成哪类状态机”
时间级别	回答时间语义到底只是局部 timer 还是强实时 / 连续耦合

sources/ 的总体统计已经足以支撑数据集和问题边界判断

统计项必须带定义列，否则数字没有解释力

前因

样本主链已经够厚，不再是“先去找样本”的阶段。

因此

所以这页要把数量和定义一起说清楚。



指标	定义	数量
论文总数	当前 sources/ 已正式入账的论文目录总数	787
论文级	单篇论文整体上可直接支持可靠样本抽取	715
论文级	单篇论文仍有价值，但需要额外整理或局部补证	16
论文级	单篇论文最终未形成可靠样本	56

类别	定义	数量
正例案例总数	正式入账、可作为控制状态样本讨论的案例条目	746
核心保留	细节和分布都足够好，适合进入主数据集主链	685
清洗后保留	有价值，但需要进一步清洗、补证或统一口径	20
降采样保留	本身可用，但因分布控制原因不应过量进入主集	41

Take-home

主链样本已经够厚；现在真正该做的是围绕它定义论文对象、baseline 口径和可执行实验。

这些比例不能被读成自然分布，它们只能证明我们已经蓄了足够厚的目标样例池

sources 是按 EFSM / HSM + T0 / T1 主动强筛出来的

前因

如果把这些比例误读成自然分布，后面的结论就会站不住。

因此

所以这一页必须明确“能说什么、不能说什么”。

不能这样解读

误读	为什么不成立
“EFSM 最多”	不能推出控制系统文献天然以 EFSM 为主；这里只能说明我们后期主动把 EFSM/HSM 样例筛得更厚。
“T0/T1 最多”	不能推出真实系统几乎没有强时间 / 连续问题；这里只能说明 project_1 当前主池刻意优先保留这两类。
“429 / 157 / 127 就是总体分布”	sources 是治理后的主样例池，不是为替整个领域做分布统计而建。

可以这样解读

可以这样说	证据	为什么重要
目标样例池已经够厚	FSM 127 / EFSM 429 / HSM 157	足以支撑 project_1 的数据集与评测设计
局部时间样例已经充足	T0 + T1 = 719 / 746	第一篇 paper 可以先收束到离散 control-state + local timing
收束对象不等于样例贫瘠	显式时钟 243 / 层次 160	目标形式主义仍需保留 hierarchy、guard 与局部 timer

结论：sources 的价值在于“样例池够厚”，不是“替整个领域做分布统计”。

这个定向样例池依然不是“简单平面 FSM 池”，它保留了层次、定时和恢复复杂度

前因

如果把目标收束误读成“只做最简单 FSM”，后面 pyfcstm 的定位也会
被说扁。

因此

所以这一页要强调池子里的结构复杂度。

主模式	定义	为什么重要	当前证据
阶段链	系统按步骤推进，每一步由 guard / timer 决定下一段。	这是离散顺序控制最常见的结构骨架。	洗衣机 PLC、电梯 PLC；T0/T1 合计 719 / 746
联锁许可	动作是否允许由权限、互斥、锁闭、到位条件共同决定。	说明 guard 与变量面不能只是附属装饰。	铁路联锁、门控控制；显式时钟 243
模式层次	上层模式管理下层子阶段、子任务或执行层。	目标 DSL 至少要保留 hierarchy。	层次样例 160；UAV supervisor 等 HSM 案例
异常恢复	fault -> safe fallback -> reset / recovery 构成专门链路。	模型不能只覆盖 happy path。	sources 中恢复链与异常保护样例长期高频出现

结构证据	数量	含义
层次	160	离散 control-state 仍大量包含父子模式关系
显式时钟	243	局部 timer / 时钟条件并不是少数样例

结论：收敛对象 = 离散 control-state，不 = 简单平面 FSM；目标形式主义至少要能承载 EFSM + HSM + local timing。

代表样本显示：真实控制系统里至少并存五类不同建模难点

同样都叫“状态机”，但它们的主难点并不相同

Framing

这些样本不是用来“凑例子”，而是用来证明“状态机”一词背后其实罩着不同问题，因此论文对象必须主动收束。

场景	标签	关键建模难点	对论文对象的启发
洗衣机 PLC	EFSM / T1	阶段链 + timer + guard	真实工业控制里“阶段 + timer”是高频对象
电梯 PLC	EFSM / T1	请求队列、方向优先、门控时长	顺序控制与门控规则构成主问题
铁路联锁	Resource / T1-T2	资源互斥、锁闭、释放	有些“状态机”本质上更像强 guard / 资源系统
UAV 分层任务	HSM / T0-T1	mission supervisor + 子层任务控制	层次任务控制是另一条高频主线
外骨骼步态控制	Hybrid / T3	连续相变量与模式切换强耦合	确实存在第一类连续 / 混成问题

结论

离散顺序控制、层次任务、资源互锁、连续耦合至少是四类不同问题。

state_machine_types 给出的启发不是“谁最多”，而是贡献点可以落在 profile / DSL / infrastructure

现代状态机研究越来越像“选择并塑造目标对象”，而不是假定唯一标准

前因

既然“状态机”本来就是家族，project_1 就不能假装目标对象天然唯一。

因此

所以必须主动选 control-state + infrastructure 这条分支。

主分支	解决的复杂度	代表对象
control-state	模式、阶段、guard、层次组织	FSM / EFSM / HSM
timed	时钟、deadline、最小 / 最大持续时间	Timed Automata / clocks
hybrid	连续动力学与离散切换耦合	Hybrid Automata / CPS
interaction-resource	资源互斥、契约组合、并发同步	Petri / Interface / contract

贡献形态	当前证据	对 project_1 的意义
DSL / target profile	59 条	目标对象设计本身就是学术贡献位点
标准 / 元模型 / 交换载体	264 条	工程上更关心可承载、可互操作、可执行
2010+ 非模型层增量	82.6%	现代增量大量发生在 runtime / bridge / workbench

结论：project_1 的贡献不一定是再发明一种“全新状态机”，也可以是主动选定并塑造一类更适合任务的 profile。

我们要解的是控制系统的离散 control-state layer ，而不是所有状态机

模式组织、联锁 guard 、异常恢复、事件权限和局部 timer 才是当前主问题

前因

“控制系统状态机”这个词太大，如果不拆开，第一篇 paper 就会同时背上离散、timed、hybrid 三类问题。

因此

所以这里先把当前最稳定、最可执行、最可验证的 control-state 语义块钉出来。

语义块	工程含义	为什么是第一篇 paper 的主问题	pyfcstm 当前承载
模式层次	系统有上层 mode 与下层子阶段 / 子任务。	控制系统最常见的结构复杂度来自 hierarchy ，而不是抽象图形语法。	可直接承载
联锁 guard	动作是否允许取决于权限、互斥、到位、锁闭等条件。	决定变量、guard 、effect 必须成为一等建模对象。	可直接承载
故障与恢复链	fault 、fallback 、reset 、recover 构成专门路径。	控制逻辑不能只覆盖 happy path 。	可直接承载
事件作用域	同一事件只在某些模式 / 边界上才有效。	这是 hierarchy 与 control-state 语义的核心交点。	可直接承载
生命周期动作	enter / during / exit 负责阶段切面与局部行为。	比“普通状态机画图”更贴近真实控制逻辑。	可直接承载
局部工程定时	延时保持、watchdog 、最小停留等局部 timer 。	第一篇 paper 需要承认 local timing ，但不必先吞下完整 TA / hybrid 家族。	部分承载；强实时仍待后续映射

结论：第一篇 paper 先解这六类 control-state 语义；连续 / 混成控制的问题承认其重要性，但放到后续延展更稳。

在 STM 语境下，pyfcstm 更像一个收窄后的 control-state DSL，而不是大而全状态机工具

最该重点比较的是 Umple / UmpleRun 这条文本状态机生态，而不是泛泛说“它像状态图”

一句话定位：sequential HSM 骨架 + EFSM 数据面 + 确定执行语义的 executable control-state DSL。

近邻对象	它在 STM 文库里做什么	与 pyfcstm 的相似点	关键差异
HSM / Statecharts	层次控制与复合状态的家族血缘	都把 hierarchy 当结构骨架	pyfcstm 主动收窄语义面，不追求完整并发 / history / 广播语义
Umple	文本化 UML / 复合状态机 DSL 与代码生成	都强调文本状态机、层次、guard / action 与可执行工件	Umple 更偏 UML 文本承载与 Java/C++ 代码生成；pyfcstm 更强调 control-state profile 与 formal core
UmpleRun	Umple 生态里的 execution-scenario 动态验证	都说明文本状态机可以直接接反馈闭环	UmpleRun 依赖 Umple -> Java/JAR -> scenario；pyfcstm 直接把 parser、runtime、symbolic core 放在同一内核里
SCXML	标准化 executable state machine / interchange 载体	都属于可执行层次状态机文本承载	SCXML 语义面更宽，含 queue、parallel、history、invoke/send；pyfcstm 更窄、更偏 control-state 闭核
Sismic	statechart runtime / testing 路线	都强调 executable state machine 与测试反馈	Sismic 更像 Python runtime；pyfcstm 更强调形式化核心与外部副作用隔离
UPPAAL	timed automata / verification backend	都和后续验证闭环强相关	UPPAAL 属于 clocks / invariants / symbolic reachability 家族；pyfcstm 当前仍是 control-state DSL 本体

结论：pyfcstm 的辨识度来自“窄 profile + 闭 formal core”，而不是“大而全兼容”。

pyfcstm 的研究价值，在于把目标形式主义、执行语义和闭环接口一起做出来

它不是附属工具，而是 project_1 的核心研究产出之一

它不是“又做了一个状态机工具”，而是在回答：应该生成什么对象、为什么它能立刻运行、以及为什么它能继续接向验证与修复闭环。

贡献点	真正回答的问题
target profile	LLM 应该生成什么对象，而不是任意 UML / SCXML 子集
executable semantics	为什么生成结果一落地就能跑起来，而不是只是一张图
formal core boundary	哪些语义留在模型核心里，哪些必须外挂给 abstract action
executable model output	输出对象为何从状态图升级成可执行形式模型
unified analysis-ready substrate	为什么它能继续接 parser / runtime / verification hooks

当前能力层	已有落地	为什么重要
parser	DSL 解析 + 结构校验	保证目标对象不是松散文本
runtime	cycle、stable-boundary、rollback	为仿真与反馈提供确定执行边界
symbolic expr	表达式到 Z3、symbolic execution	为后续 analysis-ready 路线预埋接口
codegen	Python / C 模板与生成路线	让模型能继续落向实现侧
tooling	PlantUML、文档、教程、编辑器支持	让它成为可维护对象而不是 demo

结论：pyfcstm 是 project_1 对“应该生成什么、怎样立刻可用、如何继续进闭环”的研究性回答。

project_3 的实质推进主要在 pyudbm ，但核心搜索仍缺

backend 不是空白，只是还没长成第一版 profile-guided verifier

前因

project_3 很容易被看成“目录还空，所以几乎没动”。

因此

所以这页要把“地基已做很多”和“完整 verifier 仍未成型”分开说。

a8d0649

main HEAD

本地 clone 于 ~/oo-projects/pyudbm

符号核 | UDBM API 与 UCDD 路线已立住

模型前端 | UTAP 绑定与 query 接口已接通

query + corpus | roundtrip 与 178 个官方样本已成体系

文献与路线 | TA / UPPAAL 阅读地图持续维护

层	已有	仍缺 / 影响
基础符号层	已有	不是主要瓶颈
模型 / query 接口	已有	官方样本和 roundtrip 已成体系
symbolic reachability	仍缺	无法形成真正 verifier
$A[] / E<>$ 求值	仍缺	性质检查闭环还接不上
witness / counterexample	仍缺	反馈链条不完整
verifyta 核心搜索	仍缺	这才是 project_3 真正未过的门槛

页面结论

project_3 现在最合理的定位，是继续让 pyudbm 沉淀 timed backend ，而不是抢在 project_1 之前承担主投稿。

我越来越相信基础设施反馈才是 LLM-based modeling 的真正杀手锏

parser、runtime、simulation、model checking 和 regression traceability 会把问题性质彻底改变

前因

单次生成很快会碰到 action、语义一致性和可执行性瓶颈。

因此

所以真正把质量拉高的，不是 prompt 花样，而是反馈基础设施。

2026 direct baseline

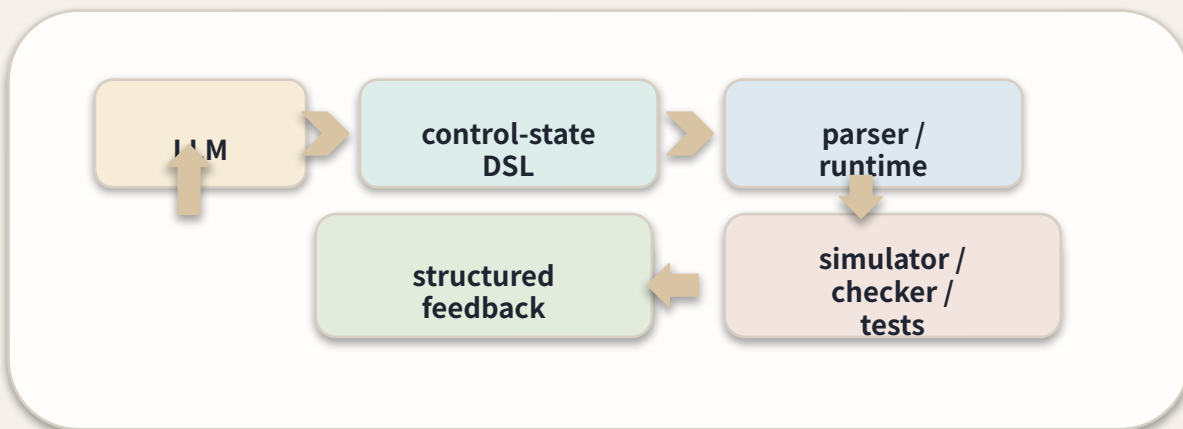
0.7029 vs 0.5431

Hybrid 流程能拉回弱模型

SysML empirical

94.6 / 88.0 / 43.1 / 37.3

规则反馈有效，但还不够



TTool AI

63 vs 58 , 15.2x

81 vs 70 , 67.5x

workflow principles

可信 GenAI 依赖
decomposition + checks +
traceability

Take-home

与其继续拼 prompt，不如把模型输出放进能持续打分、回传反例、约束格式和校验语义的环境里。

如果明天拍板这五件事，后续 6 周的推进路径会更稳

收束问题对象、拉厚实验、准备 ESEM / rolling journal 后手

前因

如果前面几项关键判断不能拍板，后面 6 周就会继续分散推进。

因此

所以这一页把倒排计划和待确认问题放在一起。

6 周倒排

第 1-2 周

收束问题定义与论文主张

第 3-4 周

固化样本与 baseline 实验

第 5-6 周

完成初稿并锁定 ESEM / journal 后手

决策 1

主投稿是否锁定 project_1

决策 2

对象是否先限于离散控制状态层

决策 3

pyfcstm 是否写成目标形式主义

决策 4

pyudbm 是否继续沉淀 backend 地基

决策 5

主稿是否按 feedback infrastructure 组织，并以 ESEM / journal 作近端出口

收束判断

只要 3-4 项能拍板，后面 6 周就能按“对象收束 -> 实验拉厚 -> 初稿成型 -> ESEM / journal 选择”推进。